

# Session-Aware Task Assistant with Long-Term Memory

Ajibade Ayomide <a.ajibade@innopolis.university>  
Ali Salloum <a.salloum@innopolis.university>

Innopolis University  
Agentic AI — Final Project Report

26 July 2026

## Abstract

We present a ReAct-style task assistant that manages interaction-derived knowledge with dual-tier memory: a bounded short-term working context (scratchpad, rolling summary, selective turn recall) and a Chroma-backed long-term store with explicit **remember/recall** tools and post-task reflection. The agent is orchestrated with LangGraph and a SQLite checkpointer, instrumented with local JSON traces and optional Langfuse, and evaluated against two in-repo baselines (stateless and full-history). We report *live* OpenRouter metrics for task quality and an *offline harness* for checker sanity and context efficiency ( $\approx 41\%$  token reduction vs. full history on a shared scenario core). Negative controls confirm that the evaluation checker rejects known failure modes.

## Contents

|          |                                                  |          |
|----------|--------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                              | <b>2</b> |
| <b>2</b> | <b>Problem and goals</b>                         | <b>2</b> |
| <b>3</b> | <b>Method</b>                                    | <b>2</b> |
| 3.1      | Agent loop . . . . .                             | 2        |
| 3.2      | Short-term memory . . . . .                      | 2        |
| 3.3      | Long-term memory . . . . .                       | 2        |
| 3.4      | Orchestration and observability . . . . .        | 3        |
| 3.5      | Baselines . . . . .                              | 3        |
| <b>4</b> | <b>Architecture</b>                              | <b>3</b> |
| <b>5</b> | <b>Memory write policy</b>                       | <b>3</b> |
| <b>6</b> | <b>Evaluation</b>                                | <b>3</b> |
| 6.1      | Offline harness (regression / checker) . . . . . | 3        |
| 6.2      | Live system metrics (OpenRouter) . . . . .       | 3        |
| <b>7</b> | <b>Discussion and limitations</b>                | <b>4</b> |
| <b>8</b> | <b>Task distribution</b>                         | <b>4</b> |
| <b>9</b> | <b>Conclusion</b>                                | <b>5</b> |

# 1 Introduction

Large language model agents are effectively stateless: each call sees only the messages supplied in the current context window. Multi-turn and cross-session work therefore loses user preferences, constraints, and prior outcomes unless the user repeats them or the system retains an unbounded transcript. Static retrieval over documents grounds answers in a corpus but not in *interaction-derived* knowledge (for example, “prefer metric units” or “we rejected design A last week”).

This project implements a lightweight assistant that executes multi-step tasks with tools while managing memory explicitly: what to write, what to recall, and how to stay within a token budget—without relying on million-token windows. The design follows the course framing of profile, memory, planning, and action; Thought–Action–Observation loops with a harness-owned observation channel; and context treated as a budget rather than a dump.

## 2 Problem and goals

### Goals.

1. Execute multi-step tasks with tools (`calculator`, `scoped search`, `remember`, `recall`, `update_scratchpad`).
2. Keep a **bounded** short-term context (scratchpad + summary + selective recall + last- $N$  turns).
3. Persist durable facts across sessions in a vector store, scoped by `user_id`.
4. Compare against (B0) stateless ReAct and (B1) append-all history using a fixed scenario suite.
5. Produce reproducible artefacts: repository, LaTeX report, and a live demo video of a real run.

**Non-goals.** Production authentication, consent productization, multi-agent orchestration, GraphRAG, MCP tool hosting, and fine-tuning are out of scope.

## 3 Method

### 3.1 Agent loop

The product path uses native tool calling. Each step the model may emit tool calls; the harness executes tools and appends observations. The loop stops on a final answer or a hard step budget (`MAX_STEPS`). The default LLM is OpenRouter `google/gemini-2.5-flash`.

### 3.2 Short-term memory

Short-term memory (STM) maintains dialogue turns and a structured scratchpad (`PLAN` / `DONE` / `NEXT`) reinjected every model call. In **bounded** mode, when the estimated token budget is approached, older turns are compressed into a rolling summary and selectively recalled by relevance; recent turns remain verbatim. In **full** mode (baseline B1), history is appended without summarization.

### 3.3 Long-term memory

Long-term memory (LTM) is a Chroma collection on disk. Writes occur via the **remember** tool (explicit user request) or a single post-task reflection pass that may propose 0–3 durable facts and must abstain when nothing lasting was learned. Near-duplicates are skipped by cosine distance. Entries carry **memory\_type** (episodic / semantic / procedural), source, and timestamps. Recall is top- $k$  and user-scoped; empty recall must surface as unknown (no confabulation).

### 3.4 Orchestration and observability

LangGraph nodes implement: assemble context  $\rightarrow$  agent  $\rightarrow$  tools (loop)  $\rightarrow$  finalize  $\rightarrow$  reflect memory. Session resume uses a SQLite checkpointer keyed by `session_id`. Every run writes a local JSON trace (input, tools, recalled IDs, token estimates, latency). When Langfuse keys are configured, the same payload is emitted as an agent observation with nested tool spans.

### 3.5 Baselines

| Mode                      | Alias | Behaviour                                             |
|---------------------------|-------|-------------------------------------------------------|
| <code>stateless</code>    | B0    | No STM history, no LTM tools/injection, no reflection |
| <code>full_history</code> | B1    | STM append-all, no LTM                                |
| <code>memory</code>       | B2    | Bounded STM + LTM + reflection                        |

## 4 Architecture

Figure 1 summarises the LangGraph control flow used in memory mode.

START  $\rightarrow$  assemble\_context  $\rightarrow$  agent  $\rightleftarrows$  tools  $\rightarrow$  finalize  $\rightarrow$  reflect\_memory  $\rightarrow$  END

Figure 1: LangGraph node pipeline (memory mode). Checkpointer: SQLite; `thread_id=session_id`.

Context assembly concatenates system instructions, scratchpad, optional LTM hits, STM history (summary + recalled turns + recent turns), and the current user task. Reflection runs only in memory mode and applies a deduplication gate before writes. Full notes live in the repository at `docs/architecture.md` and `docs/memory-policy.md`.

## 5 Memory write policy

Long-term memory is written deliberately: on explicit user request via `remember`, or after a task via reflection that may propose 0–3 durable facts (preferences, constraints, decisions) and must abstain when nothing lasting was learned. Near-duplicates are skipped so repeated preferences do not multiply. Ephemeral arithmetic and tool noise are not stored. Empty recall must be answered as unknown, never confabulated.

## 6 Evaluation

We separate two result layers and do not mix their percentages.

### 6.1 Offline harness (regression / checker)

Scripted tool calls with observation-only final answers exercise the metric checkers. Shared-core rows use equal  $N$  across B0/B1/B2. Negative controls inject missing tools, wrong search results, skipped remembers, and confabulation; the checker is expected to fail them.

Context efficiency vs. B1 on the shared core: **41.1%** mean-token reduction for memory mode (target  $\geq 30\%$ ). Negative controls: **4/4** expected failures correctly caught.

### 6.2 Live system metrics (OpenRouter)

A frozen live subset runs the real model end-to-end. These numbers support quality claims.

Table 1: Offline harness (shared-core). Not model-quality scores.

| Mode         | $N$ | Task compl. | Recall | Faithfulness | Mean tokens |
|--------------|-----|-------------|--------|--------------|-------------|
| stateless    | 10  | 100.0%      | 100.0% | 100.0%       | 1563        |
| full_history | 10  | 100.0%      | 100.0% | 100.0%       | 7033        |
| memory       | 10  | 100.0%      | 100.0% | 100.0%       | 4530        |

Table 2: Live system metrics (`google/gemini-2.5-flash` via OpenRouter).

| Mode         | $N$ | Task compl. | Recall | Faithfulness | Mean tokens |
|--------------|-----|-------------|--------|--------------|-------------|
| stateless    | 4   | 100.0%      | 100.0% | 100.0%       | 988         |
| full_history | 4   | 75.0%       | 100.0% | 100.0%       | 1098        |
| memory       | 8   | 100.0%      | 100.0% | 100.0%       | 1162        |

**Reading the tables.** Cite Table 2 for task completion / recall / faithfulness. Cite Table 1 for context efficiency and checker sanity. The live full-history STM miss (75% task completion) is a real model failure retained in the report.

## 7 Discussion and limitations

- Live  $N$  is small (frozen report subset) by design for cost and deadline; harness  $N$  is larger but scripted.
- Token estimates are heuristic; absolute token counts from the provider may differ.
- Embeddings and Chroma packaging can vary across platforms; the store is local-directory persistent.
- Reflection quality depends on the LLM; policy and dedup mitigate, but do not eliminate, weak writes.
- Langfuse is optional; local JSON traces are the always-on ground truth for debugging.

## 8 Task distribution

Table 3: Phase ownership (lead / support).

| Phase                              | Lead                    | Support         |
|------------------------------------|-------------------------|-----------------|
| P0–P1 (smoke, ReAct tools)         | Ajibade Ayomide         | Ali Salloum     |
| P2–P3 (scratchpad, context budget) | Ali Salloum             | Ajibade Ayomide |
| P4–P5 (LTM, write policy)          | Ajibade Ayomide         | Ali Salloum     |
| P6–P7 (LangGraph, observability)   | Ali Salloum             | Ajibade Ayomide |
| P8 scenarios                       | both (split categories) | —               |
| P8 harness / metrics               | Ajibade Ayomide         | —               |
| P9 report (draft)                  | Ali Salloum             | —               |
| P9 report (methods/eval)           | Ajibade Ayomide         | —               |
| P9 video (stable live run)         | Ali Salloum             | —               |

## 9 Conclusion

We delivered a session-aware ReAct assistant with bounded short-term context, cross-session long-term memory, LangGraph orchestration, dual-layer evaluation, and reproducible run traces (local and Langfuse). Live memory-mode runs meet the proposal quality targets on the frozen suite; the offline harness demonstrates context efficiency against full history and validates that the metric checker detects intentional failures.

## Repository artefacts

- Source: `src/assistant/`, scripts under `scripts/`
- Architecture / policy: `docs/architecture.md`, `docs/memory-policy.md`
- Eval: `eval/scenarios/`, `eval/suites/`, `scripts/run_eval.py`
- This report: `report/report.tex`

## References

- [1] S. Yao et al., “ReAct: Synergizing Reasoning and Acting in Language Models,” ICLR, 2023.
- [2] LangChain, “LangGraph,” documentation, 2024–2026.
- [3] Langfuse, “Open-source LLM engineering platform,” documentation, 2024–2026.
- [4] Chroma, “Chroma vector database,” documentation, 2024–2026.